MezoGig

Milestone escrow for freelance work, settled in MUSD

The client locks the whole gig budget up-front in MUSD. Each milestone is released on delivery; a silent client cannot block payment forever, and a disputed milestone is split by an arbiter.

AKINDO “Build with MUSD and MEZO” · Wave 1 · Mezo Testnet (chainId 31611)

Live demo

https://mak777uh.github.io/mezogig/

Repo: github.com/mak777uh/mezogig

Contract (Mezo Testnet)

0x247b313d0eD3C532B16c7E5f9eA760272361600F

Sourcify exact_match · 90 tests · no owner, no fee

PROBLEM

Freelancers work first and get paid later — or not at all

For the freelancer

- Deliver first → the client can disappear with the work.
- No small-claims court for a $300 gig across borders.
- Clients who do pay upfront invert the risk the other way.

For the client

- Pay first → the freelancer can disappear with the money.
- No neutral, cheap way to hold funds per milestone.

Why stablecoin, why now

- Crypto clients can pay instantly — but a volatile budget is unusable for real work.
- MUSD is a Bitcoin-backed stablecoin: the client keeps BTC exposure, the freelancer gets a stable unit.
- Existing escrow contracts are all-or-nothing: no milestones, no review window, no refunds, no arbiter.
- **Result: crypto-native work still runs on trust and screenshots.**

SOLUTION

One escrow contract, four exits, zero admin keys

1

1 · Fund

Client deposits the full budget in MUSD (approve, or one-tx EIP-2612 permit).

2

2 · Deliver

Freelancer submits a milestone with a proof link before its deadline.

3

3 · Release

Client approves → paid instantly. Silence → freelancer claims after 148 s review window.

4

4 · Exit

Missed deadline → refund. Disagreement → arbiter splits. Both agree → mutual cancel.

No owner role · no platform fee · no upgradeability · token address immutable

INTEGRATION (30 % OF THE SCORE)

How MUSD is used — not decoration, settlement

Money flows

- Deposit: safeTransferFrom(client → escrow) of the full milestone sum, or createGigWithPermit (EIP-2612) with no separate approve.
- Payout: escrow → freelancer on approve, or on claim after the review period.
- Refund: escrow → client after a missed deadline, or on mutual cancel.
- Dispute: arbiter splits the milestone by basis points; the remainder returns to the client.
- **Accounting invariant: paidOut + refunded ≤ funded, each milestone settles exactly once.**

Facts on-chain

- MUSD 0x118917a40FAF1CD7a13dB0Ef56C86De7973Ac503 — 18 decimals, permit supported.
- Contract bound to that token by an immutable address.
- Deployed on Mezo Testnet, block 16247351.

- **Honest gap: MEZO token is not integrated.**

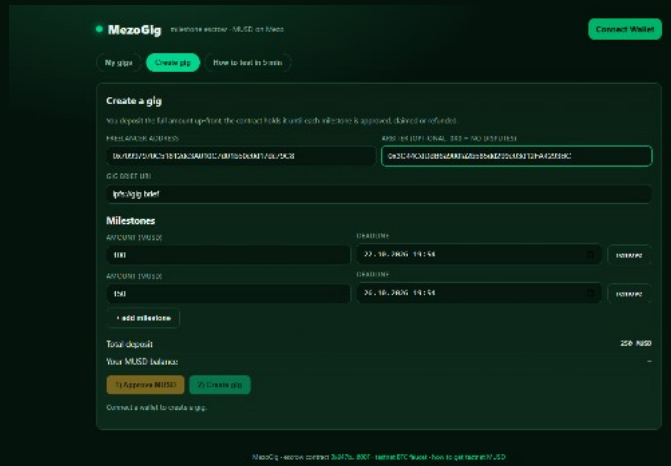
The rules allow “MUSD and/or MEZO”; MEZO utility (fee discounts, staked arbiters) is on the roadmap and is not claimed as done.

DEMO (10 % UX + 10 % MATERIALS)

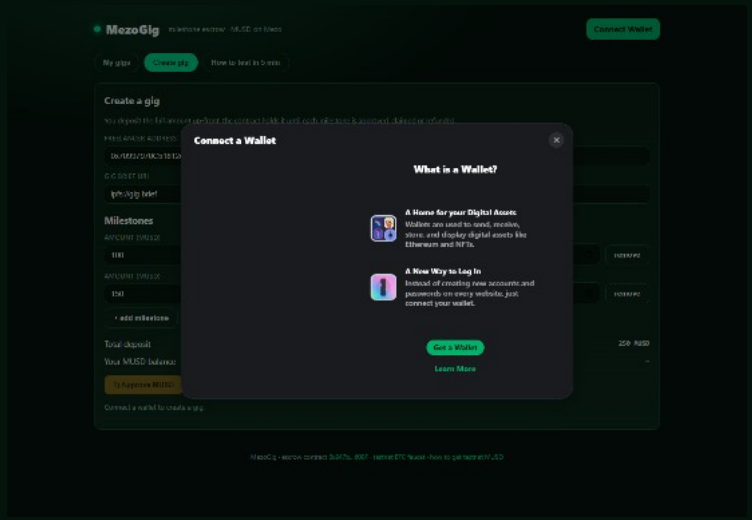
What a judge sees in the first five minutes



Gigs list, role filters, live contract link



Create: two milestones, deadlines, deposit



Wallet connect — Mezo Testnet only

Create → submit → approve → claim → dispute → refund

Live: https://mak777uh.github.io/mezogig/ · contract: https://explorer.test.mezo.org/address/0x247b313d0eD3C532B16c7E5f9eA760272361600F

TECHNICAL (20 %)

Small contract, boring security, checkable evidence

90 tests
lifecycle, guards, terminal states, fuzz invariants

100 % / 92 %
statements / branches of the escrow contract

6 attacks
malicious ERC-20 reentrancy attempts, all rejected

6 Slither findings
every one dispositioned, no high/medium

Sourcify exact_match
creation + runtime bytecode, chain 31611

Verify in one command

```
ESCROW_ADDRESS=0x247b...600F \
  node scripts/check-musd.js
→ musd(), reviewPeriod()=148, gigCount()=0

npm test        # 90 passing
npm run coverage
npm run verify:sourcify
```

Security model

No owner, no withdrawal path, no delegatecall

CEI + nonReentrant + SafeERC20, immutable token

Payouts bounded per milestone, settle once

BUSINESS (30 %)

Why the market pays for this

Who is the customer

- Freelancers and small agencies serving crypto-native clients
- DAOs outsourcing bounded tasks (design, audits, content)
- BTC holders in the Mezo economy who need to pay for work without selling BTC

Why Mezo

- MUSD is a BTC-backed stable unit — the missing piece for work payments
- Mezo's audience already holds BTC and wants to spend without exiting
- Escrow makes MUSD a settlement rail, not just a savings asset

How it makes money

- No fee in v1 (adoption); v2: optional arbiter marketplace and MEZO fee discounts
- Distribution through Mezo Discord / AKINDO builders and freelance communities
- Each gig is a repeatable on-chain flow: more MUSD volume, more wallets

Positioning: every 1 MUSD that moves through MezoGig is MUSD used for real work — the exact utility Mezo is building toward.

ROADMAP & TEAM

Three milestones, and what we are honest about

M1 · 2 weeks	Hosted frontend polish, wallet UX, gig templates, docs video
M2 · 4 weeks	Mezo mainnet deployment, third-party review, incident runbook
M3 · 8 weeks	Dispute v2: staked arbiters, reputation, MEZO fee discounts

Team

Operator (human): product, registration, payments

AI agent team: contract, tests, frontend, docs

Built during Wave 1; code, tests and deployment are public

Links

github.com/mak777uh/mezogig

mak777uh.github.io/mezogig

NEW · 23 OCT 2026

The same escrow, operable by an agent

Machine-readable CLI

scripts/agent-console.js — no UI, no keys printed

- CLI: status / jobs list / job get / watch / verify
- MCP: create_gig / fund_gig / submit / approve / dispute / status
- lifecycle commands build calldata; --dry-run is the default
- exit codes 0/1/2/3 = ok / input / network / revert
- 21 new tests (CLI + MCP + pin); npm test = 90 passing
- commit fbf6fb8; contract unchanged
- Sourcify exact_match · creation + runtime

Read-only calls checked against the deployed contract; no transaction sent.

```
$ npm run agent -- verify --json
{
  "chainId": 31611,
  "escrow": "0x247b313d0eD3C532B16c7E5f9eA760272361600F",
  "musd": "0x118917a40FAF1CD7a13dB0Ef56C86De7973Ac503",
  "musdMatches": true,
  "reviewPeriodSeconds": 148,
  "ok": true
}
$ npm test
90 passing
```

FORMAL ACCOUNTING · LEAN 4

Machine-checked invariants, honest scope

12 headline theorems · 3 groups · 0 sorry

An arithmetic model of the ledger, not a verification of Solidity bytecode.

Accounting

no_money_creation · budget_complete
dispute_split_correct · floor_split_no_gain

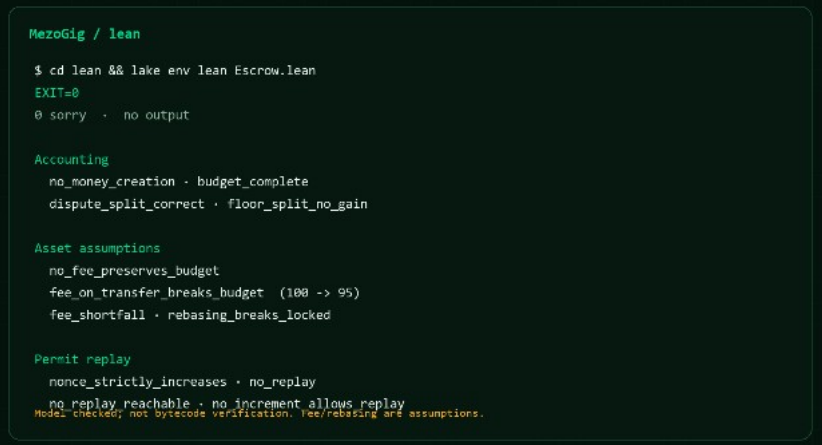
Asset assumptions

non-fee, non-rebasing ERC-20; MUSD is such a token
no_fee_preserves_budget · fee_on_transfer_breaks_budget
fee_shortfall · rebasing_breaks_locked

Permit replay

nonce_strictly_increases · no_replay
no_replay_reachable · no_increment_allows_replay

Model checked, not bytecode verification. Fee/rebasing are assumptions; if violated, the invariant fails.



lake env lean Escrow.lean → exit 0

CI: .github/workflows/lean.yml · 0 sorry

Scope: arithmetic model only; uint128 overflow, tokens and weighted splits are out of scope.